

# Personal Finance Tracker — Backend

A study guide to everything built so far, and why each decision was made.

---

**Stack:** FastAPI · SQLAlchemy 2.0 · PostgreSQL · Pydantic Settings

**Covers:** Milestone 1 (project setup) and Milestone 2 (database models & relationships)

**Generated:** 1 August 2026

## How to read this

---

Each section covers one file or one idea. Green boxes marked **Why** explain the reasoning behind a choice — those are the parts worth understanding rather than memorising, because they're what an interviewer probes. Yellow boxes marked **Gotcha** are mistakes that are easy to make and hard to debug.

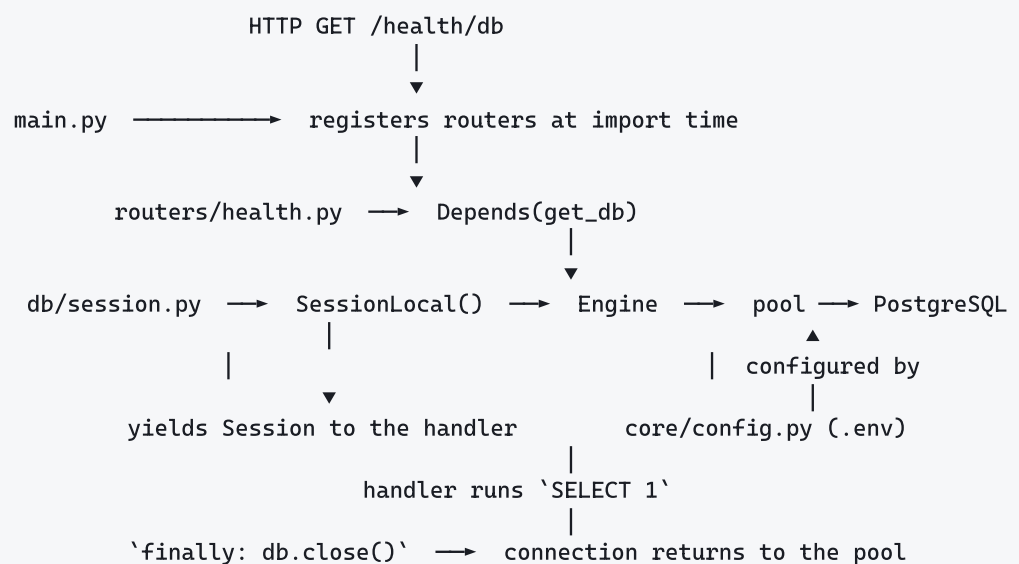
1. **The big picture** — what exists and how a request flows through it
2. **Milestone 1** — configuration, entry point, routing, folder layout
3. **Milestone 2, part A** — the database layer (Base, engine, session)
4. **Milestone 2, part B** — the four models and their relationships
5. **The generated SQL** — what all of this actually produces
6. **Verification** — how the models were proven to work
7. **Concepts worth knowing cold** — ORM ideas this project demonstrates
8. **Interview questions** — with the answers this codebase supports
9. **What's deliberately missing** — and what comes next

## 1 · The big picture

### Every file, and what it's for

| File                                   | Status    | Its one job                                                                            |
|----------------------------------------|-----------|----------------------------------------------------------------------------------------|
| <code>app/main.py</code>               | modified  | Create the FastAPI app, register routers. No logic.                                    |
| <code>app/core/config.py</code>        | modified  | All settings, typed and validated, loaded once from <code>.env</code> .                |
| <code>app/db/base.py</code>            | new       | <code>Base</code> + <code>TimestampMixin</code> . Opens no connection.                 |
| <code>app/db/session.py</code>         | new       | Engine (per process), <code>SessionLocal</code> , <code>get_db()</code> (per request). |
| <code>app/db/init_db.py</code>         | new       | Dev script: <code>python -m app.db.init_db</code> creates tables.                      |
| <code>app/models/enums.py</code>       | new       | <code>AccountType</code> , <code>TransactionType</code> , and their shared SQL types.  |
| <code>app/models/user.py</code>        | new       | The <code>User</code> model — root of every ownership chain.                           |
| <code>app/models/account.py</code>     | new       | The <code>Account</code> model — where money sits.                                     |
| <code>app/models/category.py</code>    | new       | The <code>Category</code> model — what a transaction was for.                          |
| <code>app/models/transaction.py</code> | new       | The <code>Transaction</code> model — one movement of money.                            |
| <code>app/models/__init__.py</code>    | modified  | Imports all models so <code>Base.metadata</code> is complete.                          |
| <code>app/routers/health.py</code>     | modified  | <code>/health</code> (liveness) and <code>/health/db</code> (readiness).               |
| <code>app/schemas/</code>              | unchanged | Pydantic request/response shapes. Empty until the CRUD milestone.                      |
| <code>.env.example</code>              | modified  | Committed template documenting which variables exist.                                  |

### How a request flows



## 2 · Milestone 1 — the skeleton

The guiding idea is **separation of concerns**: each folder owns one job, so a change in one layer doesn't ripple through the others.

### app/core/config.py – one place for configuration

```
class Settings(BaseSettings):
    app_name: str = "Personal Finance Tracker"
    debug: bool = False

    database_url: str | None = None      # wins if set
    postgres_user: str = "finance"      # otherwise assembled from parts
    postgres_port: int = 5432
    ...

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    @property
    def sqlalchemy_url(self) -> str:
        if self.database_url:
            return self.database_url
        return f"postgresql+psycopg2://{self.postgres_user}..."

settings = Settings()  # instantiated once, shared everywhere
```

**Why a typed `Settings` class instead of `os.environ`?** Three things you get for free. *Validation* — `postgres_port: int` means a non-numeric value fails loudly at startup instead of surfacing later as a confusing connection error. *Discoverability* — the class is the complete list of every knob the app has. *One source of truth* — nothing reads the environment scattered across the codebase, so there's never a question of where a value came from.

**Why support both `DATABASE_URL` and `POSTGRES_*` parts?** They serve different situations. The individual parts are convenient locally; a single URL is what hosting platforms, Docker Compose and CI actually hand you. Supporting both means deploying doesn't require reshaping the config. `sqlalchemy_url` is a *property* so the precedence rule lives in exactly one place — if every caller assembled the string itself, each would be a chance to get the precedence subtly wrong.

**Why `settings = Settings()` at module level?** Python caches modules, so every `from app.core.config import settings` returns the same object. The `.env` file is parsed once per process and the whole app provably agrees on what the configuration is.

### app/main.py – the entry point stays thin

```
import app.models  # noqa: F401  ← side-effect import, see below

app = FastAPI(title=settings.app_name, debug=settings.debug)
app.include_router(health.router)
```

**Why `include_router` instead of defining routes here?** Each router owns its own URLs and handlers, so adding a feature means writing a *new file* plus one line here — not growing this one into a thousand-line monolith. Note the dependency arrow: `main.py` imports routers, never the reverse. Keeping it pointing one way is what prevents circular imports as the app grows.

**Gotcha — the `import app.models` line looks unused.** It is imported purely for its *side effect*: importing the package registers every model class in SQLAlchemy's mapper registry. Without it, the first query touching a relationship fails because `Mapped["Account"]` can't be resolved to a class. A linter will flag it as an unused import, which is exactly what `# noqa: F401` silences. It does *not* create tables.

## models/ vs schemas/ — the split interviewers probe

|                    | <code>app/models/</code>    | <code>app/schemas/</code>                    |
|--------------------|-----------------------------|----------------------------------------------|
| <b>Library</b>     | SQLAlchemy                  | Pydantic                                     |
| <b>Answers</b>     | How is data <i>stored</i> ? | What does the API <i>accept and return</i> ? |
| <b>Changing it</b> | Needs a migration           | Changes the public API contract              |

**Why not use one class for both?** Because the two evolve independently. The concrete example already in this codebase: `User hashed_password` exists as a column but must never appear in a response. With the split, the response schema simply doesn't declare the field. Merge them, and every new database column silently becomes a public API change — and one day that column is a password hash.

### 3 · Milestone 2A — the database layer

#### app/db/base.py – the declarative Base

```
class Base(DeclarativeBase):
    """Shared parent of all ORM models."""

class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now(), nullable=False)
```

Subclassing `DeclarativeBase` gives every model two shared things:

- **registry** — how a string reference like `"User"` in a relationship gets resolved to the actual class.
- **metadata** — the collected table definitions that `create_all` and Alembic read.

**Why is `Base` in its own file, away from the engine?** So importing `Base` never constructs a database connection. That separation is what lets tests, migrations and tooling import the metadata without valid database settings. (This was a real fix during the build: `app/db/__init__.py` originally re-exported the session, which quietly made importing a model require a working `DATABASE_URL`.)

**Why `server_default` rather than a Python default?** `server_default=func.now()` puts `DEFAULT now()` in the actual `CREATE TABLE`, so rows written by a migration or a manual `psql` `INSERT` get timestamps too. A Python-side default only applies when the ORM does the insert — anything bypassing it leaves NULLs.

**Why a mixin instead of putting the columns on `Base`?** A mixin describes a *capability*, not a kind of thing. Models opt in by inheriting it. Put it on `Base` and every future table — including join tables and lookup tables that don't want them — is forced to carry both columns.

#### app/db/session.py – engine and sessions

```
engine = create_engine(settings.sqlalchemy_url, pool_pre_ping=True,
                       echo=settings.debug, future=True)

SessionLocal = sessionmaker(bind=engine, autoflush=False,
                             autocommit=False, expire_on_commit=False)

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

**The single most important idea here — two different lifetimes.** One `Engine` **per process**: it owns the connection pool, so it's created once at import. One `Session` **per request**: it owns a unit of work. Mixing these up is the classic SQLAlchemy mistake — a long-lived session accumulates stale objects and holds a connection open; a per-request engine throws away the pool every time, making every request pay full TCP-and-authentication cost.

| Setting                             | Why                                                                                                                                                                                           |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>pool_pre_ping=True</code>     | Cheaply checks a pooled connection is alive before handing it out. A connection dropped by Postgres (restart, idle timeout) becomes a silent reconnect instead of a random mid-request error. |
| <code>echo=settings.debug</code>    | Ties SQL logging to the existing DEBUG flag — <code>DEBUG=True</code> shows you every query the ORM emits, which is genuinely useful while learning.                                          |
| <code>autocommit=False</code>       | Transactions are explicit. You decide when work is durable.                                                                                                                                   |
| <code>autoflush=False</code>        | Stops SQLAlchemy silently emitting pending INSERTs before unrelated queries — predictable ordering, fewer surprise constraint errors mid-function.                                            |
| <code>expire_on_commit=False</code> | See the gotcha below.                                                                                                                                                                         |

**Gotcha — "Instance is not bound to a Session".** With SQLAlchemy's default `expire_on_commit=True`, every attribute is expired after `commit()`, so the next attribute read fires a fresh SELECT. If the session has already closed — which is exactly what happens when you return an ORM object from a FastAPI route — that raises. Setting it to `False` keeps attributes readable after commit and is why this error won't bite you here.

**Why does `get_db` use `try/finally` but never commit?** The `finally` guarantees the connection returns to the pool whether the route returns or raises — otherwise a single unhandled exception leaks a connection, and enough of those exhaust the pool and hang the app. It deliberately does *not* commit: a dependency shouldn't decide that a half-finished request is worth persisting. That's the handler's call.

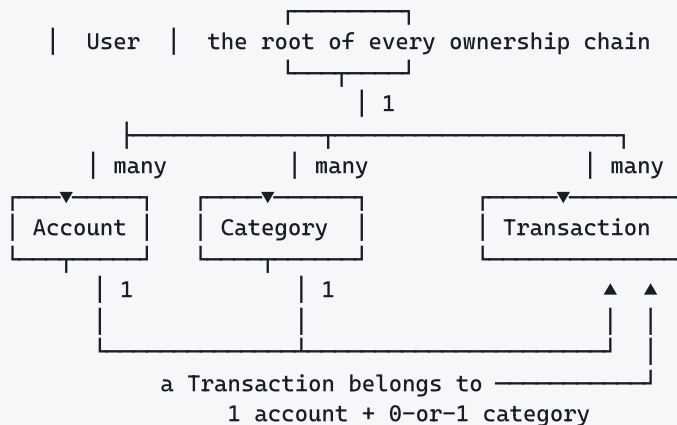
#### app/db/init\_db.py – creating the tables

```
import app.models          # side effect: populate Base.metadata
Base.metadata.create_all(bind=engine)
```

**Why a script rather than something `main.py` runs at startup?** Schema changes are a deploy-time decision. A web app that mutates the schema every time a worker boots will, under any multi-worker setup, race itself into a half-created schema.

**Gotcha — `create_all` only CREATES; it never ALTERs.** It issues `CREATE TABLE` for tables that don't exist and skips the rest. Add a column to a model and re-run it, and nothing happens — no error, no new column. That silent no-op is precisely where Alembic migrations become mandatory rather than optional.

## 4 · Milestone 2B — the four models



| Table        | Belongs to                         | Key columns                                          |
|--------------|------------------------------------|------------------------------------------------------|
| users        | —                                  | email (unique), hashed_password, is_active           |
| accounts     | user                               | name, type, balance NUMERIC(12,2), currency          |
| categories   | user                               | name, type (income/expense)                          |
| transactions | user + account + optional category | amount NUMERIC(12,2), type, occurred_on, description |

### Relationships — the mechanics

```
# On User (the "one" side)          # On Account (the "many" side)
accounts: Mapped[list[Account]] = \  owner: Mapped[User] = \
    relationship(                     relationship(
        back_populates="owner",       back_populates="accounts")
        cascade="all, delete-orphan",
        passive_deletes=True)
```

**relationship()** creates no column. The foreign key lives on the *child* table (`accounts.user_id`) — that is what "one-to-many" means physically. **relationship()** is a Python-level convenience that turns that foreign key into an attribute you can read and append to.

**The annotation declares the direction.** `Mapped[list[Account]]` (a collection) is the "one" side; `Mapped[User]` (singular) is the "many" side. SQLAlchemy reads the type hint to decide — the same function call means different things depending on the annotation.

**back\_populates** makes both ends one relationship. Naming the other side's attribute means `user.accounts.append(acct)` also sets `acct.owner`. Omit it and you get two independent relationships that can silently disagree in memory until you reload from the database.

## Cascades — declared twice, on purpose

| Where                                                         | What it does                                                                                                                                                                                                   |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>onDelete="CASCADE"</code> on the ForeignKey             | The <b>database</b> enforces it. Survives raw SQL, migrations, and any other client that touches the DB.                                                                                                       |
| <code>cascade="all, delete-orphan"</code> on the relationship | Keeps the <b>in-memory</b> object graph consistent in the current session. <code>delete-orphan</code> also means removing a child from the list deletes it, rather than leaving it pointing at nothing.        |
| <code>passive_deletes=True</code>                             | Tells SQLAlchemy to <i>trust</i> the database cascade instead of loading every child row into memory to delete them one at a time. Without it, deleting a user with 10,000 transactions issues 10,000 DELETES. |

**Why** `Category.transactions` **deliberately omits** `delete-orphan`. Deleting a category must not delete the transactions filed under it. The foreign key on the other side is `ON DELETE SET NULL`, so those rows survive as uncategorized history. This asymmetry is intentional and is the clearest example in the codebase of a cascade being a *business* decision, not a technical default.



## The decisions worth defending

1 · **Numeric, never Float, for money.** Floats are binary — `0.1 + 0.2 != 0.3` — and those fractions of a cent compound across a ledger until a balance is visibly wrong. `NUMERIC(12, 2)` is exact decimal (10 digits before the point, exactly 2 after) and maps to Python's `Decimal`. *If you remember one thing from this project, make it this one.*

2 · **Positive amount + a type enum, not signed amounts.** Direction lives in a column you can index, filter and `GROUP BY`. A `CHECK (amount > 0)` constraint then makes "negative expense" — a row that would silently *reduce* a spending total — unrepresentable rather than merely discouraged.

3 · **transactions.user\_id is denormalized.** It's reachable via `account.user_id`, so it's redundant on paper. It earns its place because ownership checks and the transaction feed are the hot path, and this turns them into one indexed scan instead of a join on every request. **Be honest about the cost:** it creates a real invariant the service layer must uphold — a transaction's user must match its account's user. Naming that trade-off is the answer; pretending there isn't one isn't.

4 · **Categories are per-user, not global.** A shared table would put one person's custom "Side Hustle" category in everyone's dropdown, and renaming it would relabel strangers' reports. Duplicated "Groceries" rows cost a few hundred bytes; the alternative leaks data between accounts.

5 · **category\_id is nullable; the other two FKs aren't.** A just-imported transaction is legitimately uncategorized, and deleting a category must not erase spending history. Hence `ON DELETE SET NULL` there, versus `ON DELETE CASCADE` for user and account.

6 · **One composite index, not three single-column ones.** `(user_id, occurred_on)` serves "my transactions, newest first" — filter and sort from one index, no sort step — *and* covers plain `WHERE user_id = ?` on its own, because an index is usable by any query constraining a **prefix** of its columns. Separate indexes on those columns would be dead weight that every INSERT still maintains.

7 · **Unique constraints are scoped to the owner.** `UNIQUE (user_id, name)` on accounts and categories: you can't have two accounts called "Chase Checking", but two different users obviously can. Scoping uniqueness to the owner is the difference between a sensible rule and a global name grab.

8 · **Date, not DateTime, for occurred\_on.** This is the day the money moved, as the bank reports it. Storing a spurious `00:00:00` invites timezone bugs where a purchase lands in the wrong month. `created_at` separately records when the *row* was written — two genuinely different facts.

**9 · Gotcha — enum SQL types must be bound to the metadata once.** On PostgreSQL an enum is a *database object*, not a column modifier. Two columns each building their own `Enum(TransactionType, name="transaction_type")` makes `create_all` emit `CREATE TYPE` twice — and the second one fails. The fix is to construct the type once with `metadata=Base.metadata` and share it:

```
transaction_type_enum = SAEnum(
    TransactionType, name="transaction_type", metadata=Base.metadata,
    values_callable=lambda c: [m.value for m in c])
```

`values_callable` stores the *values* (`"credit_card"`) rather than the Python member names (`"CREDIT_CARD"`), so the data reads sensibly in psql.

**10 · Why `str, Enum` and not plain `Enum`?** A `class TransactionType(str, Enum)` member *is* a string: it serializes straight to JSON and compares equal to `"expense"`, while still giving autocomplete and a typo-proof reference in Python.

## The circular-import problem, and how it's solved

Four models reference each other. `User` needs `Account`, `Account` needs `User` — a plain import cycle Python would reject.

```
from __future__ import annotations          # 1. all annotations become strings

if TYPE_CHECKING:                            # 2. real import for type checkers only,
    from app.models.account import Account  #   erased at runtime

accounts: Mapped[list[Account]] = relationship(...) # 3. SQLAlchemy resolves
                                                #   "Account" via the registry
```

All three layers cooperate: the type checker sees real classes, the runtime sees only strings, and SQLAlchemy looks the name up in `Base.registry` when mappers are configured. This is also why `app/models/__init__.py` importing every model matters — a class that was never imported isn't in the registry, and the lookup fails.

## 5 · The SQL this actually produces

---

Generated from the models with a mock engine — this is what `create_all` sends to PostgreSQL:

```

CREATE TYPE account_type AS ENUM ('checking','savings','credit_card','cash','investment');
CREATE TYPE transaction_type AS ENUM ('income','expense');

CREATE TABLE users (
    id SERIAL NOT NULL,
    email VARCHAR(255) NOT NULL,
    full_name VARCHAR(255),
    hashed_password VARCHAR(255) NOT NULL,
    is_active BOOLEAN NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    PRIMARY KEY (id)
);
CREATE UNIQUE INDEX ix_users_email ON users (email);

CREATE TABLE accounts (
    id SERIAL NOT NULL,
    user_id INTEGER NOT NULL,
    name VARCHAR(120) NOT NULL,
    type account_type NOT NULL,
    balance NUMERIC(12, 2) DEFAULT '0.00' NOT NULL,
    currency VARCHAR(3) DEFAULT 'USD' NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uq_accounts_user_id_name UNIQUE (user_id, name),
    FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE
);
CREATE INDEX ix_accounts_user_id ON accounts (user_id);

CREATE TABLE categories (
    id SERIAL NOT NULL,
    user_id INTEGER NOT NULL,
    name VARCHAR(80) NOT NULL,
    type transaction_type NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uq_categories_user_id_name UNIQUE (user_id, name),
    FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE
);
CREATE INDEX ix_categories_user_id ON categories (user_id);

CREATE TABLE transactions (
    id SERIAL NOT NULL,
    user_id INTEGER NOT NULL,
    account_id INTEGER NOT NULL,
    category_id INTEGER, -- nullable: uncategorized is valid
    amount NUMERIC(12, 2) NOT NULL,
    type transaction_type NOT NULL,
    description VARCHAR(255),
    occurred_on DATE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT ck_transactions_amount_positive CHECK (amount > 0),
    FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE,
    FOREIGN KEY(account_id) REFERENCES accounts (id) ON DELETE CASCADE,
    FOREIGN KEY(category_id) REFERENCES categories (id) ON DELETE SET NULL
);
CREATE INDEX ix_transactions_user_id_occurred_on ON transactions (user_id, occurred_on);

```

```
CREATE INDEX ix_transactions_account_id ON transactions (account_id);
CREATE INDEX ix_transactions_category_id ON transactions (category_id);
```

**Note — PostgreSQL does not index foreign keys automatically.** It creates an index for a PRIMARY KEY and for UNIQUE, but a plain FK column gets nothing. That's why `index=True` appears explicitly on the FK columns: without it, "all accounts for this user" is a full table scan.

## 6 · How this was verified

The models were built against a throwaway in-memory SQLite database (with `PRAGMA foreign_keys=ON`, since SQLite doesn't enforce FKs by default) so every claim above is tested rather than assumed:

| Check                                                                                 | Result                |
|---------------------------------------------------------------------------------------|-----------------------|
| All four tables present in <code>Base.metadata</code>                                 | pass                  |
| <code>back_populates</code> works in both directions                                  | pass                  |
| Four-way join User → Transaction → Account → Category                                 | pass                  |
| <code>CHECK (amount &gt; 0)</code> rejects a negative amount                          | pass — IntegrityError |
| <code>UNIQUE (user_id, name)</code> rejects a duplicate account name                  | pass — IntegrityError |
| Deleting a category leaves its transaction alive with <code>category_id = NULL</code> | pass                  |
| Deleting a user cascades all four tables to zero rows                                 | pass                  |
| <code>/health/db</code> returns 503 (not 500) when the DB is unreachable              | pass                  |
| Models import cleanly even with an invalid <code>DATABASE_URL</code>                  | pass                  |

**Not yet verified against a live PostgreSQL server** — nothing was listening on `localhost:5432`, so `python -m app.db.init_db` could not complete. Run it once your server is up; it now prints a one-line reason instead of a 60-line traceback.

## 7 · Liveness vs readiness

```
@router.get("/health")          # liveness - is the process up?
def health_check(): return {"status": "ok"}

@router.get("/health/db")       # readiness - can it reach PostgreSQL?
def db_health_check(db: Session = Depends(get_db)):
    try:
        db.execute(text("SELECT 1"))
    except SQLAlchemyError as exc:
        raise HTTPException(status_code=503, detail="database unavailable") from exc
    return {"status": "ok", "database": "connected"}
```

**Why two endpoints, and why 503 rather than 500?** Liveness answers "should this container be *restarted*?"; readiness answers "should traffic be *routed* here?". A dropped database connection is the second, not the first — collapse them and a brief DB blip gets your perfectly healthy app killed and restarted. `503` says "the app is fine, its dependency isn't", which is what tells a load balancer to route elsewhere and retry; `500` would say the app itself is broken.

## 8 · Interview questions this codebase answers

---

### Q. Why not store money as a float?

A. Floats are binary and can't represent most decimal fractions exactly — `0.1 + 0.2 != 0.3`. Across a ledger those errors compound into a visibly wrong balance. `NUMERIC(12,2)` is exact decimal and maps to Python's `Decimal`.

### Q. What's the difference between a SQLAlchemy model and a Pydantic schema?

A. The model describes how data is *stored*; the schema describes what the API *accepts and returns*. Keeping them separate lets an internal column like `hashed_password` exist without ever appearing in a response, and lets the DB shape and the API contract evolve independently.

### Q. What does `back_populates` actually do?

A. It links the two ends of one relationship so assigning either side updates both in memory. Without it you have two independent relationships that can silently disagree until you reload from the database.

### Q. You set the cascade in two places. Isn't that redundant?

A. No — they cover different layers. `onDelete` on the ForeignKey is the database enforcing it, which survives raw SQL and other clients. `cascade` on the relationship keeps the in-memory object graph consistent within the session. `passive_deletes=True` then lets the DB do the bulk work instead of SQLAlchemy loading every child row.

### Q. Why is `user_id` on transactions when you could join through accounts?

A. It's deliberate denormalization. Ownership checks and the transaction feed are the hot path, and this makes them a single indexed scan rather than a join on every request. The cost is an invariant the service layer must maintain — the transaction's user must match its account's user.

### Q. Why does one foreign key allow NULL when the others don't?

A. Nullability encodes a business rule. A transaction must have an owner and an account, but "uncategorized" is a legitimate state — and deleting a category shouldn't erase spending history, so that FK is `ON DELETE SET NULL` while the others cascade.

### Q. Why one composite index instead of indexing each column?

A. An index is usable by any query constraining a *prefix* of its columns, so `(user_id, occurred_on)` already covers `WHERE user_id = ?`. A separate index on `user_id` would add nothing while still costing write time on every INSERT.

### Q. How does the app avoid circular imports between models that reference each other?

A. Three things together: `from __future__ import annotations` makes all annotations strings; the real imports sit under `if TYPE_CHECKING` so they're erased at runtime; and SQLAlchemy resolves the string name through `Base.registry`. That's also why `app/models/__init__.py` imports every model — an unimported class isn't in the registry.

**Q. Why is the engine created once but the session created per request?**

A. They own different things. The engine owns the connection pool, which should be shared and long-lived. The session owns a unit of work, which should be short-lived and isolated — a long-lived session accumulates stale objects and holds a connection open.

**Q. Why not call `create_all()` on application startup?**

A. Schema changes are a deploy-time decision, and multiple workers booting at once would race into a half-created schema. It's also insufficient — `create_all` never ALTERs an existing table, so the first column change silently does nothing. That's where Alembic takes over.

## 9 · What's deliberately missing — and what's next

---

There is no service/repository layer, no dependency-injection framework, and no Alembic yet. A project of this size doesn't need those abstractions, and adding them early is over-engineering. The structure leaves clean seams to introduce them *if* the project grows. **Being able to say why you stopped here is as much the point as the structure itself.**

The honest caveat: `create_all` gets you a schema, not schema *evolution*. The moment a column changes on a database holding real rows, Alembic stops being optional.

### LIKELY NEXT MILESTONES

1. **Pydantic schemas + CRUD endpoints** — fills the empty `schemas/` folder and finally exercises `get_db` for real reads and writes.
2. **Alembic migrations** — replaces `init_db.py` as the way the schema changes.
3. **JWT auth** — gives `hashed_password` and `is_active` their purpose, and turns every `user_id` column into an enforced authorization check.
4. **Docker** — one `DATABASE_URL` pointing at a Postgres container, which is exactly why `config.py` supports the single-URL form.